DESENVOLVIMENTO MOBILE · MÓDULO 19 · TRILHA FLUTTER · NÍVEL AVANÇADO

Código Nativo, Web, Desktop e Embarcados

Canais de plataforma, Pigeon, FFI, criação de plugins, Flutter dentro de apps nativos, Flutter web, desktop e Raspberry Pi

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Comparar as abordagens multiplataforma e justificar uma escolha.
- Chamar código Kotlin e Swift a partir do Dart com canais de plataforma e Pigeon.
- Usar bibliotecas C com FFI.
- Criar um plugin próprio e embutir telas Flutter em apps nativos.
- Publicar o app na web e no desktop, adaptando o layout a cada tela.
- Executar um painel Flutter num Raspberry Pi.

Roteiro

Neste módulo

- Fundamentos do desenvolvimento multiplataforma
- A arquitetura do Flutter
- Canais de plataforma
- FFI: bibliotecas em C
- Plugins e Flutter em apps nativos
- Flutter na web
- Desktop e embarcados
- Fechamento

Fundamentos do desenvolvimento multiplataforma

O problema: muitas plataformas, uma equipe

Por que existe o multiplataforma

Android e iOS têm linguagens, ferramentas e componentes diferentes. Dois apps nativos significam duas bases de código, duas equipes e o dobro de testes para cada funcionalidade.

Desenvolvimento multiplataforma

Escrever uma vez parte (ou todo) o código e executá-lo em mais de um sistema, aceitando algum compromisso em desempenho, aparência ou acesso ao hardware.

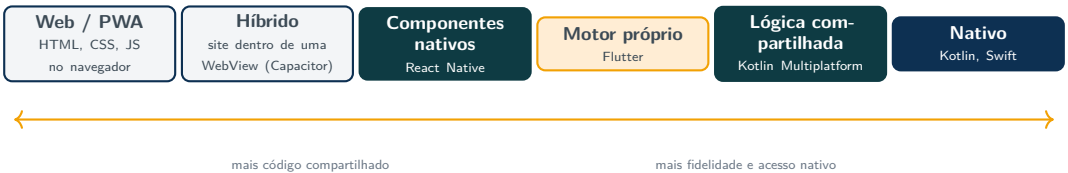
Um pouco de história

Java prometia "escreva uma vez, rode em qualquer lugar" nos anos 1990. No mobile, vieram os apps híbridos com WebView (PhoneGap/Cordova, 2009), o React Native (2015), o Flutter (2017–2018) e o Kotlin Multiplatform (estável em 2023).

Atenção

Nenhuma abordagem elimina o conhecimento de cada plataforma: permissões, publicação, ciclo de vida e integrações nativas continuam diferentes.

O espectro das abordagens



	O que se compartilha	Quem desenha a tela	Compromisso principal
Web/PWA e híbrido	tudo	o navegador (HTML/CSS)	desempenho e acesso limitado ao hardware
React Native	UI e lógica em JS/TS	componentes nativos de cada sistema	ponte entre JS e o nativo
Flutter	UI e lógica em Dart	o motor do Flutter, pixel a pixel	tamanho do app; aparência própria
KMP	lógica (e opcionalmente a UI)	nativa ou Compose Multiplatform	UI pode exigir código por plataforma

Como o código executa em cada abordagem

Tecnologia	Execução no aparelho
PWA / híbrido	JavaScript interpretado e compilado sob demanda (JIT) pelo motor do navegador
React Native	JavaScript pré-compilado em *bytecode* e executado pelo motor Hermes
Flutter	Dart compilado antes (AOT) para código de máquina ARM
KMP (Android)	Kotlin → *bytecode* → DEX, executado pelo ART
KMP (iOS)	Kotlin/Native compilado via LLVM para binário nativo
Nativo	Kotlin no ART; Swift compilado via LLVM

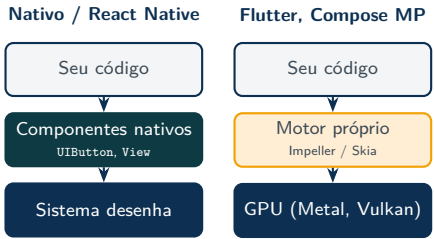
JIT e AOT

JIT (*just-in-time*): compila durante a execução; flexível, permite recarregar código. **AOT** (*ahead-of-time*): compila antes da distribuição; inicia mais rápido e tem desempenho previsível.

Por que o iOS importa aqui

A App Store não permite que apps gerem e executem código de máquina novo em tempo de execução (exceto o motor do navegador). Por isso o Flutter usa AOT no iOS e o React Native usa um interpretador de *bytecode*.

Quem desenha os pixels?



Componentes nativos

A tela usa os controles do sistema: aparência e comportamento idênticos aos de outros apps e atualizados junto com o sistema.

Motor próprio

O framework desenha cada pixel numa tela em branco: aparência idêntica em todas as plataformas e controle total de animações, mas precisa reimplementar o comportamento de cada controle.

Analogia

Montar a casa com móveis de loja (nativos) ou fabricar os próprios móveis (motor próprio).

Conversando com o nativo: pontes, canais e FFI

Mecanismo	Como funciona
Troca de mensagens	chamadas serializadas e assíncronas entre os dois lados (canais de plataforma do Flutter, ponte antiga do React Native)
Interface direta	chamada de funções nativas sem serializar, inclusive síncrona (FFI do Dart, JSI do React Native)
Geração de código	a interface é declarada uma vez e o código dos dois lados é gerado (Pigeon, Turbo Modules, exportação para Swift do KMP)

O custo de atravessar

Cada chamada por mensagem converte os dados para bytes, troca de *thread* e converte de volta. Para chamadas raras (abrir a câmera) não importa; para milhares por segundo (áudio, imagens), use interfaces diretas.

FFI — *Foreign Function Interface*

Mecanismo para chamar funções escritas em outra linguagem, em geral bibliotecas C, compartilhando memória.

Progressive Web Apps

PWA

Site que se comporta como app: pode ser instalado na tela inicial, funciona offline e recebe notificações. Base: um **manifesto** (nome, ícones, cores) e um ***service worker*** (script que intercepta requisições e gerencia o cache).

Vantagens

Uma única base web, sem loja e sem revisão, atualização instantânea, link compartilhável.

Limites

- Acesso a hardware depende do navegador: Web Bluetooth e Web NFC não existem no Safari.
- No iOS, notificações exigem que o usuário instale o site na tela inicial.
- Menos presença nas lojas e menos integração com o sistema.

Dica

Para um painel de consulta do LabMonitor no computador do laboratório, uma versão web basta; para BLE e NFC, o app nativo ou multiplataforma é necessário.

Decidindo com uma matriz ponderada

Critério	Peso	Nativo	Flutter	RN	KMP
Atender Android e iOS	3	2	5	5	4
Acesso a BLE, NFC, câmera	3	5	4	4	5
Equipe já sabe a linguagem	2	4	3	2	4
Tempo até a 1ª versão	2	2	5	4	3
Manutenção em 5 anos	1	4	4	3	4
Total ponderado		37	**47**	42	45

Notas de 1 a 5 ilustrativas: app novo, equipe que conhece Kotlin. Com um app Kotlin já existente, o critério "reaproveitar código" favoreceria o KMP.

Matriz de decisão ponderada

Lista os critérios, dá um peso a cada um conforme sua importância, atribui notas às opções e soma nota × peso.

Atenção

O resultado depende dos pesos: discuta-os com a equipe e o cliente **antes** de dar as notas, para não ajustar a matriz ao resultado desejado.

Além do celular: telas e formatos

Responsivo e adaptativo

Responsivo: o mesmo layout se estica ou reorganiza conforme o espaço. **Adaptativo**: layouts diferentes para classes de tela e para cada forma de entrada (toque, mouse, teclado, controle remoto).

Classe	Largura	Exemplos
Compacta	< 600 dp	celular em pé
Média	600–839 dp	tablet em pé, dobrável aberto
Expandida	≥ 840 dp	tablet deitado, desktop

Classes de janela do Material Design.

Outros formatos

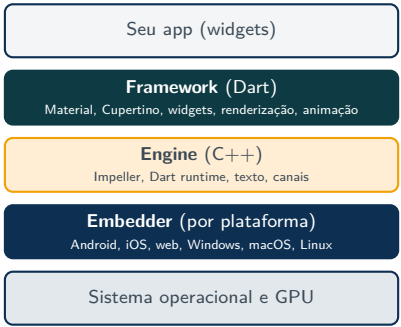
Relógios, TVs, carros (Android Auto, CarPlay), painéis embarcados e quiosques. Cada um muda a distância de leitura, a forma de entrada e o tempo de atenção do usuário.

Dica

Projete pelo **espaço disponível**, não pelo tipo de aparelho: uma janela de desktop pode ser estreita como um celular.

A arquitetura do Flutter

Camadas: framework, motor e *embedder*



Quem faz o quê

Framework: tudo que você usa em Dart.
Engine: desenha, roda o Dart, faz o *layout* de texto e transporta as mensagens dos canais.
Embedder: cria a janela ou superfície, entrega toques e teclado e integra com o ciclo de vida de cada sistema.

Portabilidade

Para levar o Flutter a uma plataforma nova, basta escrever um *embedder*. É assim que ele roda em painéis de carros e no Raspberry Pi.

Canais de plataforma

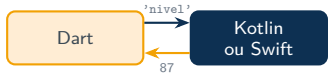
`MethodChannel`: o lado Dart

lib/data/services/bateria_nativa.dart

```dart
import 'package:flutter/services.dart';

class BateriaNativa {
  static const _canal =
      MethodChannel('br.edu.ifma.labmonitor/bateria');

  Future<int> nivel() async {
    try {
      final n = await _canal.invokeMethod<int>('nivel');
      return n ?? -1;
    } on PlatformException catch (e) {
      throw BateriaIndisponivel(e.message);
    } on MissingPluginException {
      return -1;            // plataforma sem implementação
    }
  }
}
```



Canal

Nome único + *codec*. A mensagem (nome do método e argumentos) é convertida em bytes, atravessa para o nativo e a resposta volta como `Future`.

Dica

Tipos suportados pelo *codec* padrão: `null`, `bool`, números, `String`, listas, mapas e `Uint8List`.

MethodChannel: os lados Kotlin e Swift

android/.../MainActivity.kt

```kotlin
class MainActivity : FlutterActivity() {
  override fun configureFlutterEngine(
      engine: FlutterEngine) {
    super.configureFlutterEngine(engine)
    MethodChannel(engine.dartExecutor.binaryMessenger,
        "br.edu.ifma.labmonitor/bateria")
      .setMethodCallHandler { call, result ->
        if (call.method == "nivel") {
          val bm = getSystemService(BATTERY_SERVICE)
              as BatteryManager
          result.success(bm.getIntProperty(
              BatteryManager.BATTERY_PROPERTY_CAPACITY))
        } else result.notImplemented()
      }
  }
}
```

ios/Runner/CanalBateria.swift

```swift
func registrarCanalBateria(
    _ mensageiro: FlutterBinaryMessenger) {
  let canal = FlutterMethodChannel(
    name: "br.edu.ifma.labmonitor/bateria",
    binaryMessenger: mensageiro)
  canal.setMethodCallHandler { call, result in
    guard call.method == "nivel" else {
      return result(FlutterMethodNotImplemented)
    }
    UIDevice.current.isBatteryMonitoringEnabled =
      true
    result(Int(UIDevice.current.batteryLevel * 100))
  }
}
```

Dica

Registre o canal onde o `AppDelegate` obtém o *messenger* do motor.

EventChannel e Pigeon

Fluxo contínuo do nativo

```dart
const _eventos = EventChannel(
    'br.edu.ifma.labmonitor/luz');

Stream<double> luminosidade() =>
    _eventos.receiveBroadcastStream()
        .map((v) => (v as num).toDouble());
```

EventChannel

O nativo envia valores quando quiser (sensor, conexão); no Dart, chegam como `Stream`.

pigeons/sensor_api.dart

```dart
import 'package:pigeon/pigeon.dart';
class Leitura {
  Leitura({required this.valor, required this.emMs});
  double valor;
  int emMs;
}

@HostApi()
abstract class SensorNativoApi {
  @async
  Leitura ler(String sensor);
}
```

```
$ dart run pigeon --input pigeons/sensor_api.dart
```

Pigeon

Gera Dart, Kotlin e Swift tipados a partir da interface.

FFI: bibliotecas em C

Chamando C a partir do Dart

src/filtro.c

```c
double media_movel(const double *v,
                   int n) {
  double s = 0;
  for (int i = 0; i < n; i++)
    s += v[i];
  return n > 0 ? s / n : 0;
}
```

Quando usar

Reaproveitar bibliotecas C/C++ ou acelerar cálculos chamados com muita frequência.

Dica

`ffigen` gera as assinaturas a partir do `.h`.

lib/utils/filtro.dart

```dart
import 'dart:ffi';
import 'package:ffi/ffi.dart';
typedef _MediaC = Double Function(Pointer<Double>, Int32);
typedef _MediaDart = double Function(Pointer<Double>, int);

final _lib = DynamicLibrary.open('libfiltro.so');  // Android, Linux
final _media = _lib.lookupFunction<_MediaC, _MediaDart>('media_movel');

double mediaMovel(List<double> valores) {
  final p = calloc<Double>(valores.length);   // memória nativa
  try {
    for (var i = 0; i < valores.length; i++) p[i] = valores[i];
    return _media(p, valores.length);
  } finally {
    calloc.free(p);                           // o GC não libera
  }
}
```

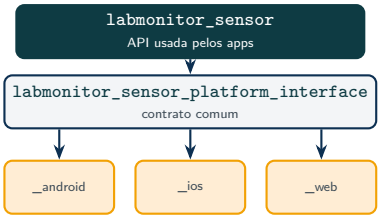
Atenção

Memória nativa não é coletada pelo GC: libere sempre.

Plugins e Flutter em apps nativos

Criando um plugin

```
$ flutter create --template=plugin \
    --platforms=android,ios \
    --org br.edu.ifma labmonitor_sensor
```



Plugin federado

Divide o plugin em pacotes: a API pública, uma interface de plataforma e uma implementação por plataforma. Outras pessoas podem implementar plataformas novas sem mudar o pacote principal.

Plugin × pacote

Pacote contém só Dart. **Plugin** tem também código nativo para uma ou mais plataformas.

Dica

Antes de criar um plugin, procure no pub.dev: provavelmente alguém já resolveu o problema.

Add-to-app: Flutter dentro do app Kotlin

```
$ flutter create --template=module relatorios_flutter
```

App Android nativo (Kotlin)

```kotlin
class LabApp : Application() {
  override fun onCreate() {
    super.onCreate()
    // motor pré-aquecido: a tela abre sem atraso
    val motor = FlutterEngine(this).apply {
      dartExecutor.executeDartEntrypoint(
        DartExecutor.DartEntrypoint.createDefault())
    }
    FlutterEngineCache.getInstance().put("relatorios", motor)
  }
}

// ao tocar em "Relatórios"
startActivity(FlutterActivity
    .withCachedEngine("relatorios").build(this))
```

Add-to-app

Incorporar telas Flutter a um app nativo existente, como módulo, em vez de reescrevê-lo inteiro.

Cenário

O LabMonitor em Kotlin ganha a tela de relatórios em Flutter, compartilhada com o iOS (onde se usa um `FlutterViewController`).

Atenção

Cada motor ocupa memória: reaproveite motores em cache ou use `FlutterEngineGroup`.

Flutter na web

Compilando para o navegador

```
$ flutter run -d chrome
$ flutter build web --wasm --release
# publicar num subcaminho (GitHub Pages)
$ flutter build web --base-href /LabMonitor/
```

Renderizadores

CanvasKit: o Skia compilado para WebAssembly, desenhando num *canvas*.
skwasm: versão mais rápida, usada com `-wasm` quando o navegador suporta WebAssembly com coleta de lixo.

Quando não usar

Sites de conteúdo que dependem de busca (SEO), como blogs e páginas institucionais: o Flutter web desenha num *canvas* e baixa um motor inteiro. Ele brilha em **aplicações**: painéis, ferramentas internas, versões web de apps.

Dica

Hospedagem simples: Firebase Hosting, GitHub Pages ou qualquer servidor de arquivos estáticos.

Código que roda em todo lugar

Diferenças de plataforma

```dart
import 'package:flutter/foundation.dart';
// importação condicional: escolhe o arquivo na compilação
import 'arquivos_io.dart'
    if (dart.library.js_interop) 'arquivos_web.dart';

// decisão em tempo de execução
final suportaBle = !kIsWeb &&
    (defaultTargetPlatform == TargetPlatform.android ||
     defaultTargetPlatform == TargetPlatform.iOS);
```

Por que importação condicional

`dart:io` (arquivos, *sockets*) não existe no navegador. Cada plataforma recebe o arquivo que sabe usar, com a mesma interface.

Política de mesma origem e CORS

O navegador bloqueia requisições de `labmonitor.dev` para `api.outro.dev`, a menos que a API responda com o cabeçalho `Access-Control-Allow-Origin` autorizando a origem.

Funciona no celular, falha na web

Apps nativos não aplicam CORS; o navegador sim. A correção é **no servidor** (Módulo 17), não no app.

Desktop e embarcados

Desktop e layout adaptativo

```
$ flutter create --platforms=windows,macos,linux .
$ flutter run -d linux
$ flutter build linux --release
```

Um painel para cada largura

```dart
@override
Widget build(BuildContext context) => LayoutBuilder(
  builder: (context, c) => switch (c.maxWidth) {
    < 600 => const PainelCompacto(),     // lista
    < 840 => const PainelMedio(),        // grade
    _ => const PainelExpandido(),        // lista + detalhe
  },
);
```

O que muda no desktop

- Mouse: *hover*, clique direito, rolagem.
- Teclado: atalhos (`Shortcuts`, `Actions`) e foco com Tab.
- Janelas redimensionáveis e menus.
- Instaladores próprios: MSIX, DMG, Snap ou DEB.

Dica

`switch` com padrões relacionais (Dart 3) deixa as faixas de largura legíveis.

Painel do laboratório num Raspberry Pi

```
# no computador de desenvolvimento
$ flutter pub global activate flutterpi_tool
$ flutterpi_tool build --arch=arm64 --release
# copiar build/flutter-pi para o Pi e rodar
$ flutter-pi --release ~/labmonitor
```

flutter-pi

Embedder leve que roda o Flutter direto na placa de vídeo do Raspberry Pi, sem ambiente gráfico: inicia rápido e ocupa a tela inteira, ideal para quiosques.

LabMonitor em modo quiosque

Um Raspberry Pi 4 ou 5 com tela de toque fixado na parede mostra as bancadas, as reservas do dia e os alertas, recebendo dados por MQTT (Módulo 13).

Hardware da placa

GPIO, I^2C e SPI podem ser acessados via FFI ou pacotes como `dart_periphery`: o painel lê sensores ligados diretamente ao Pi.

Um pouco de história

Montadoras já usam Flutter em centrais multimídia de carros, um dos usos mais conhecidos de embarcados.

Fechamento

Resumo

Resumo

- As abordagens vão do web/híbrido ao nativo; o Flutter compartilha tudo e desenha com motor próprio.
- Framework, engine e *embedder* separam o Flutter das plataformas.
- Canais de plataforma trocam mensagens; Pigeon gera o código tipado; FFI chama C diretamente.
- Plugins federados e *add-to-app* integram Flutter e código nativo nos dois sentidos.
- Web, desktop e embarcados reaproveitam o código, com layout adaptativo e atenção às diferenças (CORS, `dart:io`).

Glossário (1/2)

Termo	Significado
Multiplataforma	código escrito uma vez e executado em mais de um sistema
JIT / AOT	compilação durante a execução / antes da distribuição
Engine	núcleo em C++ do Flutter: renderização, Dart *runtime*, texto e canais
Embedder	camada que integra o *engine* a uma plataforma específica
Canal de plataforma	meio de troca de mensagens entre Dart e código nativo
Codec	regra que converte valores em bytes e de volta para atravessar o canal
Pigeon	gerador de código tipado para os dois lados de um canal
FFI	interface para chamar funções C a partir do Dart, compartilhando memória

Glossário (2/2)

Termo	Significado
Plugin federado	plugin dividido em API, interface de plataforma e implementações
Add-to-app	incorporação de telas Flutter a um app nativo existente
PWA	site instalável, com manifesto e *service worker*
WebAssembly	formato binário executado pelos navegadores com desempenho próximo ao nativo
CORS	mecanismo do navegador que controla requisições entre origens diferentes
Layout adaptativo	layouts distintos por classe de tamanho de janela e forma de entrada
Quiosque	aparelho dedicado a um único app, em tela cheia
Matriz ponderada	técnica de decisão que soma notas multiplicadas por pesos

Questões de revisão

1. Compare React Native e Flutter quanto a quem desenha os componentes da tela. Que consequência isso tem para a aparência do app?
2. Por que o Flutter compila AOT no iOS?
3. Qual a diferença entre um canal de plataforma e o FFI? Dê um caso de uso para cada um.
4. Que problema o Pigeon resolve em relação ao `MethodChannel` escrito à mão?
5. O que é um plugin federado e qual a vantagem dessa organização?
6. Por que uma chamada HTTP pode funcionar no app Android e falhar no Flutter web?
7. Quando o Flutter web **não** é uma boa escolha?
8. Monte uma matriz ponderada para decidir entre Flutter e PWA para um painel de consulta do laboratório.

Exercícios

Exercício 1 — Canal de plataforma

Crie um `MethodChannel` que devolve o nome do modelo do aparelho no Android e no iOS, tratando a ausência de implementação.

Exercício 2 — FFI

Compile a função `media_movel` em C, chame-a pelo Dart e compare o tempo com uma versão em Dart para 10 000 amostras.

Exercício 3 — Web e desktop

Gere as versões web e desktop do LabMonitor, tornando o painel adaptativo para as três classes de largura.

LabMonitor — Etapa 19

Painel em modo quiosque (web ou Raspberry Pi) recebendo leituras por MQTT, e um canal de plataforma próprio para uma função nativa que nenhum plugin oferece.

Para aprofundar

- Arquitetura do Flutter: `https://docs.flutter.dev/resources/architectural-overview`
- Canais de plataforma: `https://docs.flutter.dev/platform-integration/platform-channels`
- FFI: `https://dart.dev/interop/c-interop`
- *Add-to-app*: `https://docs.flutter.dev/add-to-app`
- Flutter web: `https://docs.flutter.dev/platform-integration/web`
- flutter-pi: `https://github.com/ardera/flutter-pi`

Respostas das questões de revisão

1. O React Native usa os componentes nativos de cada sistema, que se parecem com os de outros apps; o Flutter desenha os próprios, com aparência idêntica nas plataformas e independente da versão do sistema.
2. A App Store não permite gerar e executar código de máquina novo em tempo de execução; com AOT, o código já chega compilado.
3. O canal troca mensagens serializadas e assíncronas com Kotlin/Swift (abrir uma API do sistema); o FFI chama funções C diretamente, inclusive de forma síncrona (filtro de sinais com muitas chamadas).
4. Elimina nomes de métodos em texto e conversões manuais: a interface declarada gera código tipado nos dois lados, e erros aparecem na compilação.
5. É um plugin dividido em API pública, interface comum e uma implementação por plataforma; novas plataformas podem ser adicionadas por pacotes separados.
6. O navegador aplica a política de mesma origem: sem o cabeçalho CORS na resposta da API, a requisição é bloqueada.
7. Em sites de conteúdo que dependem de SEO e de carregamento muito rápido da primeira página.
8. Exemplo de critérios: acesso a BLE (peso 1, pois o painel só consulta), instalação sem loja (3), custo de manutenção (2), desempenho de gráficos (2). A PWA tende a vencer se o painel apenas exibe dados.

Fim do Módulo 19

Próximo: Módulo 20 — Tópicos avançados e projeto final: IA, segurança, monetização e o projeto integrador.